REPORT

SRN: PES2UG24CS601

Phase 1:

Screenshot 1A:

```
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ make test_objects
make: 'test_objects' is up to date.
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B:

```
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2:

Screenshot 2A:

```
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ make test_tree
make: 'test_tree' is up to date.
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B:

```
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ xxd .pes/objects/f5/902ee73b6a
a25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2 | head -20
00000000: 7472 6565 2039 3800 3130 3036 3434 2066  tree 98.100644 f
00000010: 696c 6531 2e74 7874 002c f8d8 3d9e e295  ile1.txt.,..=...
00000020: 43b3 4a87 7274 21fd ecb7 e3f3 a183 d337  C.J.rt!........7
00000030: 6390 25de 576d b9eb b431 3030 3634 3420  c.%.Wm...100644
00000040: 6669 6c65 322e 7478 7400 e00c 50e1 6a2d  file2.txt...P.j-
00000050: f38f 8d6b f809 e181 ad02 48da 6e67 19f3  ...k......H.ng..
00000060: 5f9f 7e65 d6f6 0619 9f7f                 _.~e......
```

Phase 3:

Screenshot 3A:

```
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ echo "hello" > file1.txt
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ echo "world" > file2.txt
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ ./pes add file1.txt file2.txt
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.h
  untracked:  tree.h
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  Makefile
  untracked:  object.c
  untracked:  index.h
  untracked:  pes.c
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  commit.c
  untracked:  test_objects
  untracked:  pes.h
  untracked:  index.c
  untracked:  .gitignore
  untracked:  test_tree.c
```

Screenshot 3B:

```
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776437012 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776437021 6 file2.txt
```

Phase 4:

Screenshot 4A:

```
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 55ce52d0b0f5... Initial commit
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ ./pes log
commit 55ce52d0b0f580f4cdd2169c17d0b820193b41fea00a84e8858f9011b3d42fef
Author: Vrinda Venugopal M  <PES2UG24CS601>
Date:   1776437268

    Initial commit
```

Screenshot 4B:

```
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/55/ce52d0b0f580f4cdd2169c17d0b820193b41fea00a84e8858f9011b3d42fef
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2
.pes/refs/heads/main
```

Screenshot 4C:

```
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ cat .pes/refs/heads/main
55ce52d0b0f580f4cdd2169c17d0b820193b41fea00a84e8858f9011b3d42fef
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Full integration test:

```
vrinda@vrinda-VirtualBox:~/PES2UG24CS601-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: e73a8a39ddcf... Initial commit

Log after first commit:
commit e73a8a39ddcf85f3827c4d2353e3c6d4dd224b81ed755e23d7b66f2179986c41
Author: Vrinda Venugopal M  <PES2UG24CS601>
Date:   1776438310

    Initial commit


--- Second Commit ---
Committed: 16434bdc5062... Update file.txt

--- Third Commit ---
Committed: eafde064bb23... Add farewell

--- Full History ---
commit eafde064bb23a8de3d708dc3576d350f62569f4fa985d585710aa77955223465
Author: Vrinda Venugopal M  <PES2UG24CS601>
Date:   1776438311

    Add farewell
```

```
commit 16434bdc506294cf243f754529b18e6bc18396c1b4b934867b64e977e8ad7c27
Author: Vrinda Venugopal M  <PES2UG24CS601>
Date:   1776438310

    Update file.txt

commit e73a8a39ddcf85f3827c4d2353e3c6d4dd224b81ed755e23d7b66f2179986c41
Author: Vrinda Venugopal M  <PES2UG24CS601>
Date:   1776438310

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
eafde064bb23a8de3d708dc3576d350f62569f4fa985d585710aa77955223465

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/16/434bdc506294cf243f754529b18e6bc18396c1b4b934867b64e977e8ad7c27
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e7/3a8a39ddcf85f3827c4d2353e3c6d4dd224b81ed755e23d7b66f2179986c41
.pes/objects/ea/fde064bb23a8de3d708dc3576d350f62569f4fa985d585710aa77955223465

=== All integration tests completed ===
```

Phase 5:

5.1:

To implement pes checkout <branch>, first we need to update what HEAD points to. In .pes/HEAD, instead of pointing to the current branch, we change it to point to refs/heads/<branch>. Then we read the commit hash stored in that branch file.

After getting the commit, we need to reconstruct the working directory to match that commit's tree. That means reading the tree object of that commit, writing all files and directories from the tree into the working directory and updating the .pes/index to match this new state

The complex part is safely replacing the working directory. If files already exist or have been modified, blindly overwriting them can cause data loss. Also, handling nested

directories and ensuring consistency between index, objects, and working directory makes the operation tricky.

5.2:
To detect a dirty working directory, we compare the current working files with what is stored in the index. The index represents the last staged state, so it acts like a reference.

For each tracked file - check its metadata (mtime and size) against the index, if different, recompute the hash and compare with the stored blob hash if hashes don't match, the file has uncommitted changes

Then, we also check the target branch's tree. If the same file exists there but has a different version, switching branches would overwrite local changes. In that case, checkout should refuse.

So basically, the conflict is detected by comparing working directory, index, target branch tree.
If all three don't align safely, we block the checkout.

5.3:
In a detached HEAD state, HEAD directly stores a commit hash instead of pointing to a branch. If we make new commits in this state, they still get created normally, but no branch is pointing to them.

This means those commits can become lost because there is no reference to reach them later. However, they still exist in the object store temporarily.

To recover them, the user can either create a new branch pointing to that commit using the hash or use logs (like reflog in real Git) to find the commit and restore it

So detached HEAD is not dangerous by itself, but if you don't save your commits by creating a branch, they can eventually be removed during cleanup.

Phase 6:

6.1:

To delete unreachable objects, we first need to find all reachable ones. We start from all branch heads (like files in .pes/refs/heads/) and mark their commit hashes as reachable.

Then we recursively traverse:

- Each commit - its tree and parent commits
- Each tree - its blobs and subtrees

We can use a hash set (like a set of ObjectIDs) to keep track of all reachable objects efficiently.

After marking everything reachable, we scan the object store and delete any object not in the set.

For a repo with 100,000 commits and 50 branches, we don't visit 100k × 50 separately. Many commits overlap between branches, so total unique objects might still be around 100k commits plus their trees and blobs. So overall, we might visit a few hundred thousand objects, not millions.

6.2:

Running garbage collection (GC) during a commit is dangerous because of race conditions. For example, suppose a commit is being created and has written a tree object but hasn't yet updated the branch reference.

At the same time, GC runs and checks which objects are reachable. Since the new commit isn't yet referenced by any branch, GC may think those objects are unused and delete them.

After that, when the commit process tries to finish, it ends up referencing objects that no longer exist, leading to corruption.

Real Git avoids this by using safety mechanisms like not deleting very recent objects, using locks during operations and running GC only at safe times

This ensures that partially written commits are not accidentally removed.